MEMORIA TÉCNICA ARIMA

DATASET DE KAGGLE

En este caso he elegido un dataset acerca del precio de las acciones de Microsoft, donde se muestra:

- La fecha diaria de las acciones (excepto los fines de semana)
- Acción última del día (close)
- La primera acción del día (open)
- La acción más baja del día (low)
- La acción más alta (high)

He elegido este dataset ya que los modelos ARIMA se puede utilizar también para predecir acciones en bolsa y veía interesante hacer un modelo para poder predecir las acciones futuras de una empresa (Microsoft en este caso)

PASOS SEGUIDOS

Una vez elegido el dataset, vamos a cargar el csv en R y tratar los datos

➔ Paso 2: Ingesta y Conversión a Serie Temporal

Una vez cargado el csv, haremos una serie de transformaciones, en este caso, como el dataset tiene columnas del valor de la bolsa en el cierre, apertura y los valores más altos y bajos, lo que he hecho es hacer una media entre todos ellos y crear una columna llamada "trading_price", siendo nuestra columna objetivo.

“microsoft_stock_df <- microsoft_stock_df %>% mutate(trading_price = (Close + High + Low + Open)/4)”

Por otro lado, después de estudiar el dataset y viendo videos acerca del funcionamiento en profundidad de ARIMA, este dataset va desde el año 1986 hasta 2025, y en el tramo 1986-2020 hay muchos valores cercanos a 0. Luego hay un cambio brusco ascendente en 2020. Si a eso lo juntamos con que el modelo ARIMA es muy delicado con los datos donde haya alta volatilidad, es decir, donde hay cambios bruscos, he sacado la conclusión de que los datos cercanos a 0 no es de utilidad en este caso, ya que solo hace que haya ruido en el modelo, debido a que a día de hoy, excepto sorpresas, es muy difícil que el valor en bolsa de Microsoft esté cercano a 0, por lo que hacemos un filtrado para eliminar los datos desde 1984 hasta 2020

“microsoft_stock_df <- microsoft_stock_df %>% filter(Date >= as.Date("2020-01-01"))”

Por otra parte, las fechas están en tipo carácteres, por lo que lo pasamos a tipo Date con la librería “lubridate”

“microsoft_stock_df$Date <- ymd(microsoft_stock_df$Date)”

Para terminar este paso, convertimos la variable objetivo en una serie temporal (ts), donde establecemos una frecuencia de 252 (ya que las fechas de las acciones son diarias excepto los fines de semana y festivos)

“trading_price <- ts(microsoft_stock_df$trading_price, start = c(2020,1,1), frequency = 252)”

➔ Paso 3: Descomposición

Separamos la serie en tendencia, estacionalidad y residuos con la función decompose()

➔ Paso 4: Análisis de Estacionariedad

En este paso, comprobamos si nuestra serie tiene estacionalidad con el test de Dickey-Fuller Aumentado, para ello hacemos uso de la función adf.test de la librería tseries. Una vez sepamos el resultado (comentado posteriormente), en caso de que sea necesario diferenciar la serie, verificamos cuantas diferenciaciones hay que hacer con la función ndiffs()

➔ Paso 5: Entrenamiento del Modelo

Una vez diferenciado la serie, hacemos dos modelos arima y lo comparamos entre ellos, un modelo se hará de forma automática con la función auto.arima(), y otro modelo donde ajustamos los parámetros manualmente. Viendo la gráfica acf y pacf podemos deducir los valores de p y q. Una vez tenemos ambos modelos lo comparamos viendo los valores del impacto de los errores (ma) y el valor de AIC

➔ Paso 6: Validación del Modelo

Una vez elegido el modelo correspondiente, analizamos los residuos con la función checkresiduals(), donde podemos ver si se sigue una distribución normal, también se hace el test Ljung-Box, donde si el p-value es mayor que 0.05, por lo que los datos son independientes (ruido blanco)") y no tienen autocorrelación significativa, en caso contrario, el modelo no se ha ajustado a los datos de la serie. Además, con el atributo $residuals de la serie y la función mean(), podemos calcular la media residual y comprobar que sea 0.

➔ Paso 7: Forecasting

Por último, una vez ajustado el modelo, podemos hacer predicciones, donde en este caso, he hecho predicciones de los próximos 500 días y se ha mostrado el intervalo de confianza

Todos estos resultados, se verán en el punto que viene a continuación

➔ Train/Test Split

Vamos a dividir los datos en dos partes, todos los datos excepto los últimos 90 días (3 meses), lo vamos a guardar en en una variable que usaremos para entrenar el modelo. Luego las últimas 90 filas (90 días), lo guardamos en una variable para el test. Una vez dividido, creamos un modelo ARIMA, con los datos de entrenamiento y hacemos predicciones. Por último comparamos gráficamente los datos de la predicción y los de test

➔ Comparativa con Prophet

Creamos un dataframe a partir del primero pero solo con la fecha y el precio de la acción promedia del día, ya que Prophet solo necesita esas dos variables. Luego entrenamos ese modelo con el data frame creado haciendo uso del método prophet(). Por último se hace las predicciones y se muestra gráficamente

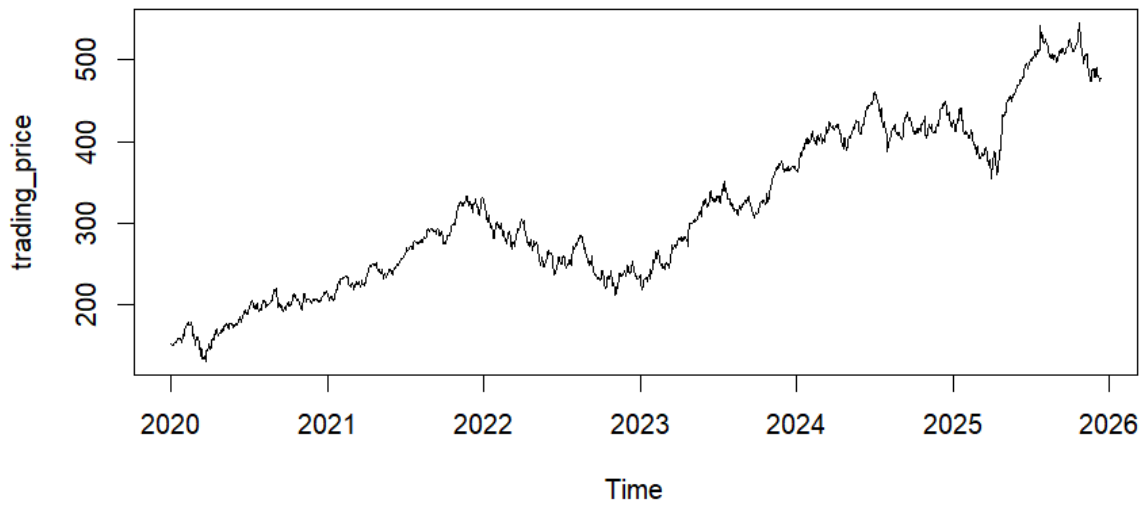
➔ Detección de Anomalías

En este último caso, detectamos los outliers con la función tsoutliers(), luego creamos un data frame con los outliers obtenidos y pintamos una gráfica con los outliers con un color azul. En este caso, al parecer no se ha detectado ningún outlier

DOCUMENTACIÓN DE PRUEBAS REALIZADAS

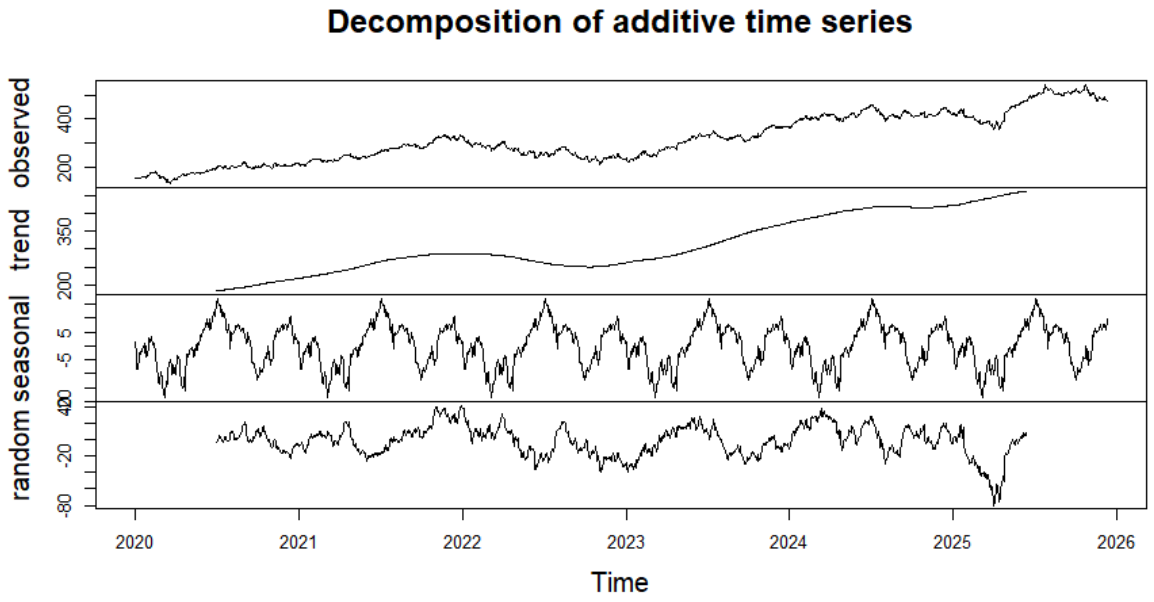
En este punto se mostrarán los resultados obtenidos en cada paso (gráficas, valores numéricos…), así como la configuración que se ha hecho para obtener dichos resultados

En primer lugar empezamos con la carga de datos, en el que, una vez obtenida la serie temporal, podemos visualizarlo gráficamente:



En la gráfica podemos observar que hay una tendencia clara ascendente, con algunos picos de bajada o de subida. Sin embargo, no se puede observar un patrón claro. Aunque posteriormente, al descomponer la serie si podemos ver en la gráfica estacionaria un patrón

Ahora pasamos al paso 3, donde separamos la serie en tendencia, estacionalidad y residuos, vemos gráficamente las gráficas ya separadas



Si observamos la gráfica estacional, podemos concluir que hay un patrón que se repite cada año, y es que al iniciar el año, suele caer el valor de las acciones, y en verano suele subir bastante las acciones

Luego pasamos con el paso 4, donde aplicamos el test de Dickey-Fuller con la función adf.test, donde tenemos:

- H0: La serie no es estacionaria
- H1: La serie es estacionaria

Vemos que nos sale estos resultados:

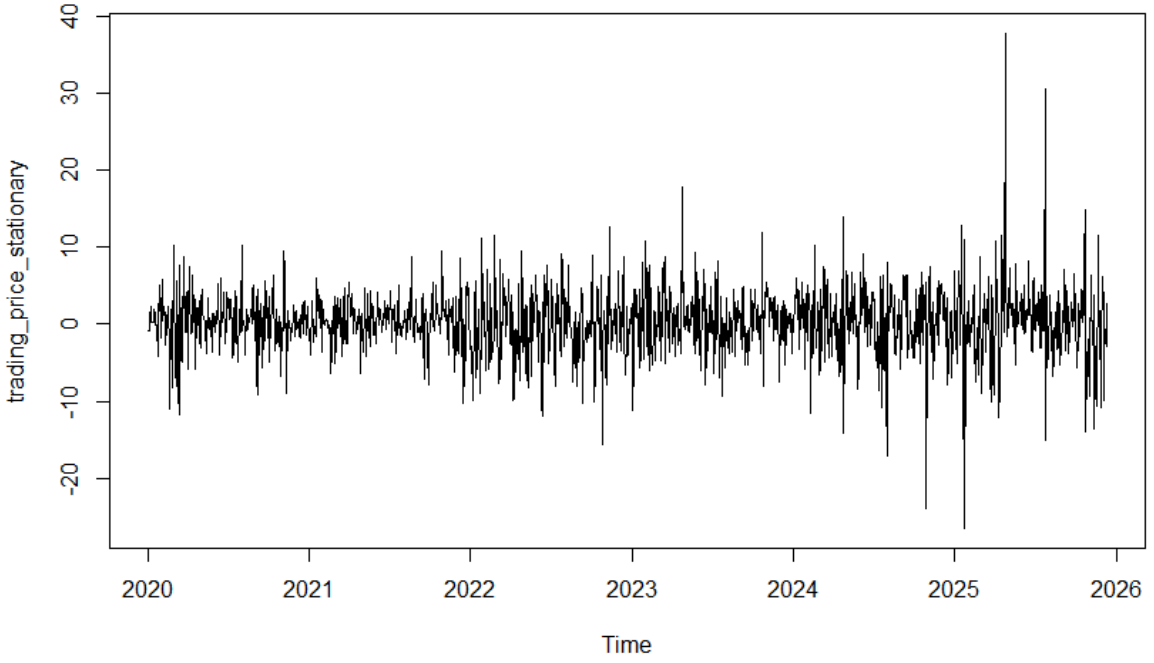
```
Augmented Dickey-Fuller Test


data:  trading_price

Dickey-Fuller = -2.3666, Lag order = 11, p-value = 0.4231

alternative hypothesis: stationary
```

Observamos que el p-value es bastante mayor que 0.5, por lo que no podemos rechazar la hipótesis nula (H0), por lo que la serie no es estacionaria, teniendo que diferenciar la serie para que lo sea y podamos usar el modelo ARIMA

Para ver cuantas diferencias son necesarias, hacemos uso de la función ndiffs(), y vemos que nos sale que tenemos que hacer solo una diferenciación

```
> n_diferencias <- ndiffs(trading_price) # Con ndiffs vemos cuántas diferenciación hay que hacer
> paste("Hay que hacer ", n_diferencias, " diferencias (d)")
[1] "Hay que hacer  1  diferencias (d)"
```

Para diferenciar la serie hacemos uso de la función diif() con el parámetro differences a 1 y vemos que así queda gráficamente



Una vez diferenciado pasamos al quinto paso donde usaremos el modelo ARIMA. Primero entrenamos un modelo ARIMA automático, donde intentará obtener los parámetros p, d y q más óptimos que encuentre, basándose en el valor AIC (que lo usaremos para comparar ambos modelos). Al entrenar el modelo obtenemos que los parámetros óptimos es una p con valor 0, la diferenciación (d) es 1 y el valor de q es 2. Estos son los resultados obtenidos:

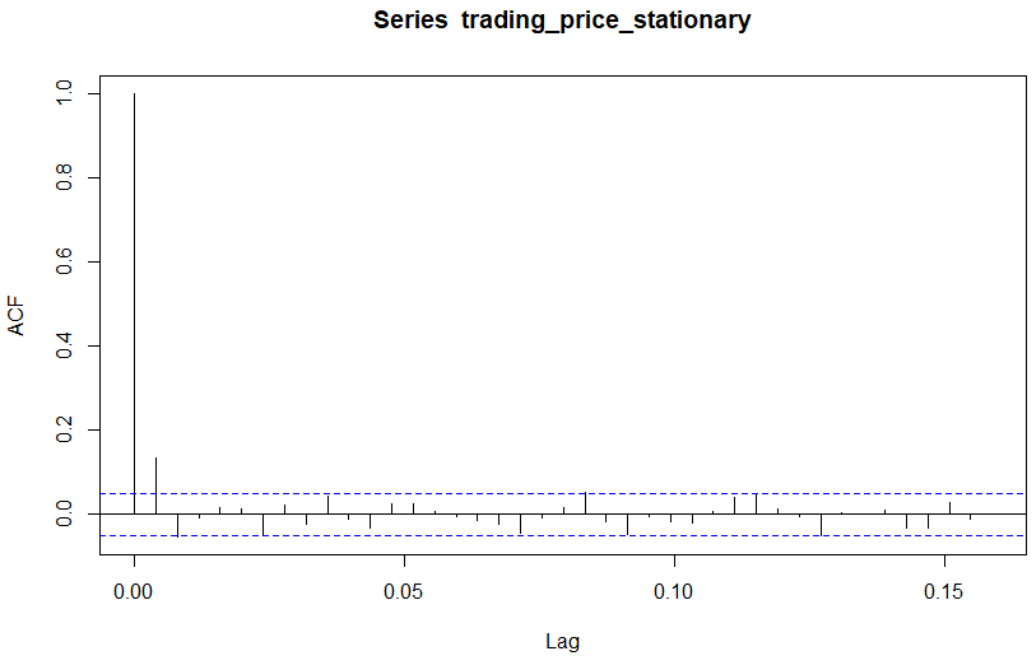
```
> summary(arima_automático)
Series: trading_price
ARIMA(0,1,2) with drift

Coefficients:
         ma1      ma2   drift
      0.1454  -0.0506  0.2173
s.e.  0.0258   0.0260  0.1181

sigma^2 = 17.48:  log likelihood = -4267.03
AIC=8542.05   AICc=8542.08   BIC=8563.3

Training set error measures:
                       ME     RMSE      MAE          MPE     MAPE       MASE         ACF1
Training set 0.0003044498 4.175358 2.921552 -0.009290584 0.995332 0.03999975 0.0004571459
> coeftest(arima_automático)

z test of coefficients:

       Estimate Std. Error z value  Pr(>|z|)
ma1    0.145439   0.025831  5.6304 1.798e-08 ***
ma2   -0.050615   0.026022 -1.9451   0.05176 .
drift  0.217314   0.118145  1.8394   0.06586 .
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

Con summary vemos los valores de ma1 y ma2, que representan el impacto de los errores de uno y dos periodos anteriores respectivamente, y los utiliza para recalibrar las predicciones, el primero (ma1) tiene un valor de 0.1454, por lo que el periodo anterior afecta ligeramente al valor actual, y el segundo (ma2) tiene un valor muy bajo de -0.0506, por lo que el error de hace dos periodos afecta muy poco. Esto hace que el modelo corrija el valor actual usando errores recientes. También podemos observar los valores de los errores, donde vemos que son valores aceptables, por ejemplo, RMSE (nos permite medir la diferencia promedio entre valores predichos y reales) tiene un valor de 4 aproximadamente, cosa que está bastante bien, luego tenemos el ME (Mean Error), que es el error promedio y es muy cercano a 0, por lo que es buena señal o el MAE, que como tiene un valor cercano a 3, quiere decir que se equivoca en 3 unidades de precio.

Por otra parte, con la función coeftest() podemos ver como de importantes son los diferentes coeficientes, donde vemos que la primera es la más importante y es muy significativa, y la segunda no lo es tanto

Por otro lado, vamos a analizar los correlogramas ACF y PACF de la serie estacionaria para saber los valores de p y q.

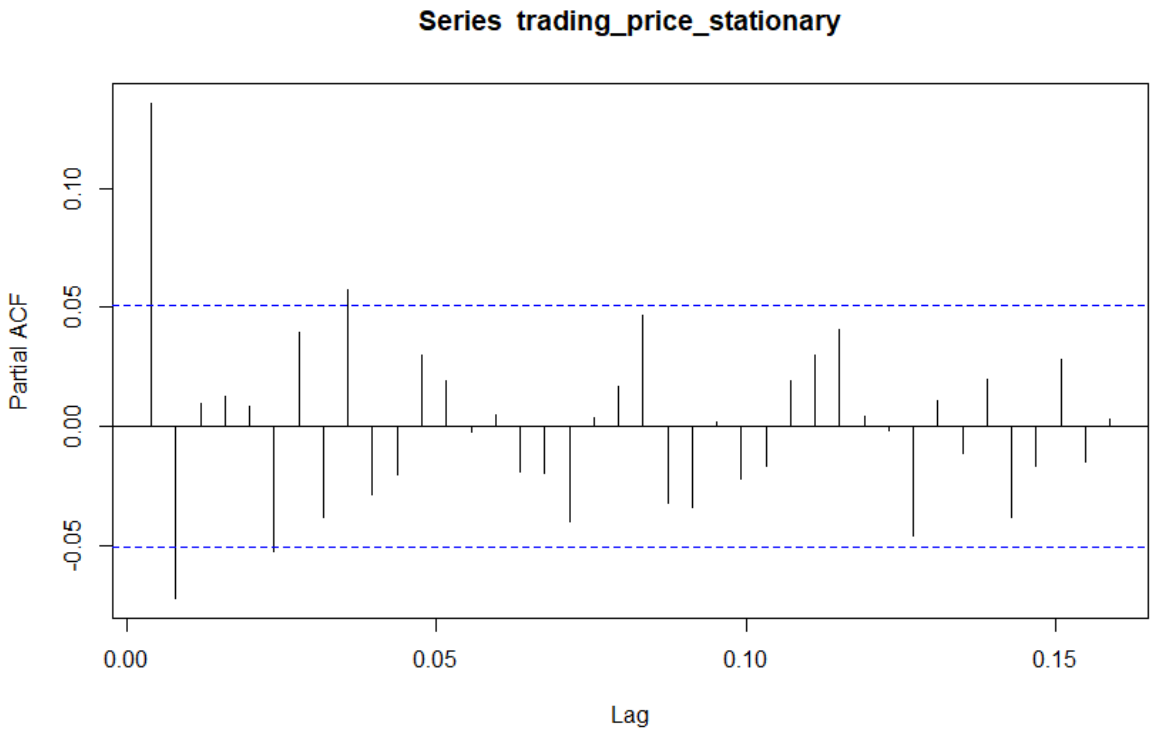
Veamos en primer lugar el correlograma ACF:



Si observamos la tabla de la [página web adjunta](#) a la actividad

Forma	Modelo
Serie exponencial decayendo a 0	Modelo Auto Regresivo (AR). Función pacf () que se utilizará para identificar el orden del modelo
Picos alternativos positivos y negativos, decayendo a 0	Modelo Auto Regresivo (AR). Función pacf () que se utilizará para identificar el orden del modelo
Uno o más picos en serie, resto todos son 0	Modelo de media móvil (MA), identifica el orden donde el gráfico se convierte en 0
Después de algunos retrasos en general la serie va decayendo.	Modelo mezclado AR & MA
La serie total es 0 o casi 0	Datos aleatorios
Valores medios a intervalos fijos	Necesitamos incluir el término AR de estacionalidad
Picos visibles que no decaen a 0	Series no son estacionarias

Vemos que nos encontramos con el tercer caso, donde hay uno o más picos en la serie, y el resto es 0. Una vez sabemos esto, miramos la gráfica PACF para saber el orden o valor del parámetro q



En la tabla nos dice que nos fijemos cuando el gráfico se convierte en 0 (entre las barras azules) y vemos que se convierte en 0 entre el 0.02 y el 0.04, por lo que probaremos entre ese rango de valores (2-4) para obtener un modelo ARIMA óptimo

Una vez hecho varias pruebas, he visto que el mejor valor para el parámetro q es el valor 2, a continuación veamos los resultados obtenidos:

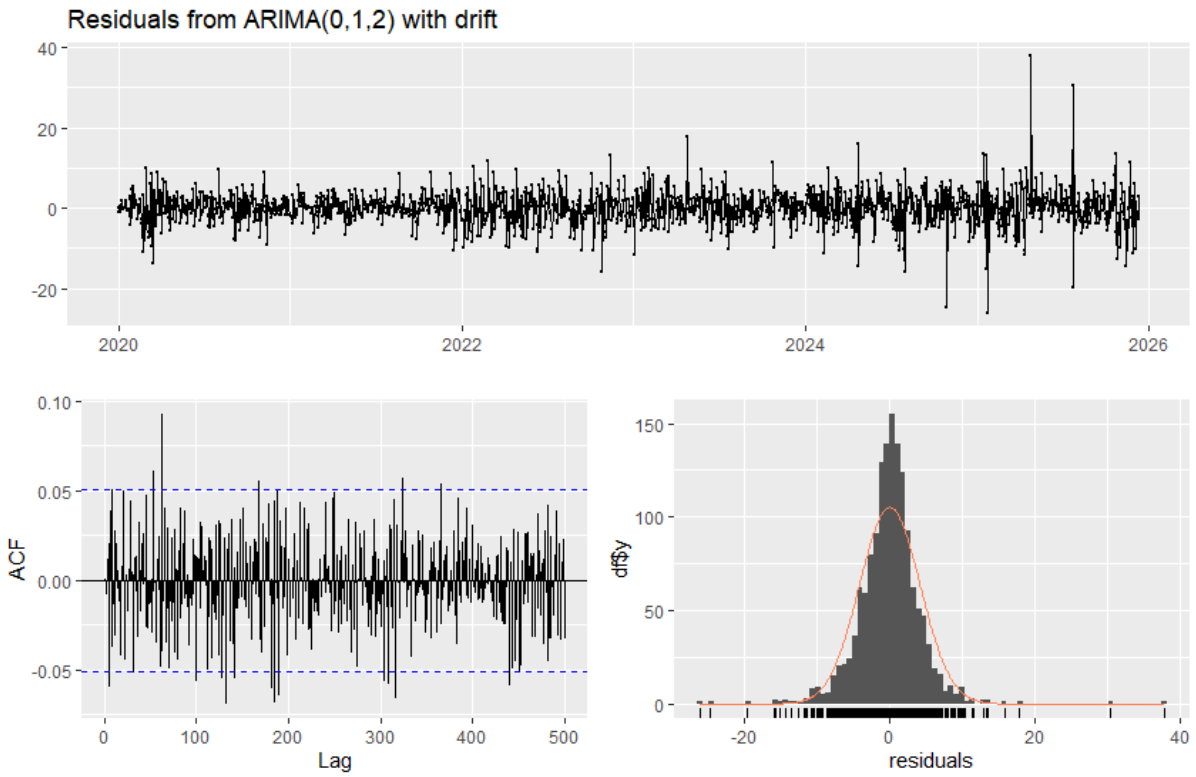
```
> summary(trading_arima)

Call:
arima(x = trading_price, order = c(0, 1, 2), method = "ML")

Coefficients:
         ma1      ma2
      0.1478  -0.0482
s.e.  0.0258   0.0260

sigma^2 estimated as 17.48:  log likelihood = -4268.71,  aic = 8543.42

Training set error measures:
                    ME     RMSE      MAE        MPE      MAPE       MASE         ACF1
Training set 0.1978122 4.180058 2.932578 0.06046286 0.9991989 0.04015071 -0.001891571
> coeftest(trading_arima)

z test of coefficients:

     Estimate Std. Error z value  Pr(>|z|)
ma1  0.147817   0.025822  5.7244 1.038e-08 ***
ma2 -0.048218   0.025958 -1.8576   0.06323 .
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

Vemos que obtenemos unos valores de ma1 y ma2 muy parecidos al anterior modelo, igual pasa con los valores de los errores. Algo parecido ocurre con la importancia de los coeficientes, aunque en este caso, el valor de Pr(>|z|) de ma1 (que mide la importancia del coeficiente) es algo menor en este caso, por lo que es mejor pero el valor de ma2 es mayor.

Sin embargo, la clave está en fijarse en el valor AIC (Criterio de información de Akaike) que aparece en el summary(), y es que, es una medida de la calidad relativa de un modelo estadístico, para un conjunto dado de datos, y en este caso, cuanto menor sea ese valor, mejor es el modelo. Ambos valores son muy parecidos, aunque el valor de AIC del modelo automático es un poco menor que en el modelo manual (no obstante la diferencia es muy pequeña), así que, para ser justos y seguir con el enunciado, elegimos en este caso el modelo automático

Una vez elegido el modelo, pasamos con el paso 6, donde se hace la validación del modelo. En este caso, vamos a hacer un análisis de los residuos del modelo. Para ello, aplicamos la función checkresiduals(), donde nos aparece ciertas gráficas que analizaremos a continuación. También, con la función mencionada, hace el test de Ljung-Box

A continuación tenemos las gráficas que se generan cuando ejecutamos checkresiduals():



En primer lugar podemos concluir que hay una distribución normal (como vemos en el tercer gráfico), luego podemos observar la primera gráfica, que simplemente son los residuos del modelo y la segunda es el correlograma del modelo

Ahora vamos a ver el test que se ha realizado:

```
> residuos <- checkresiduals(arima_automático)

        Ljung-Box test

data:  Residuals from ARIMA(0,1,2) with drift
Q* = 329.53, df = 298, p-value = 0.1011

Model df: 2.   Total lags used: 300
```

En primer lugar hay que aclarar que el test de Ljung-Box tiene dos hipótesis:

- ➔ H0: Los datos son independientes (ruido blanco); no hay autocorrelación significativa
- ➔ H1: Los datos muestran autocorrelación, no son aleatorios

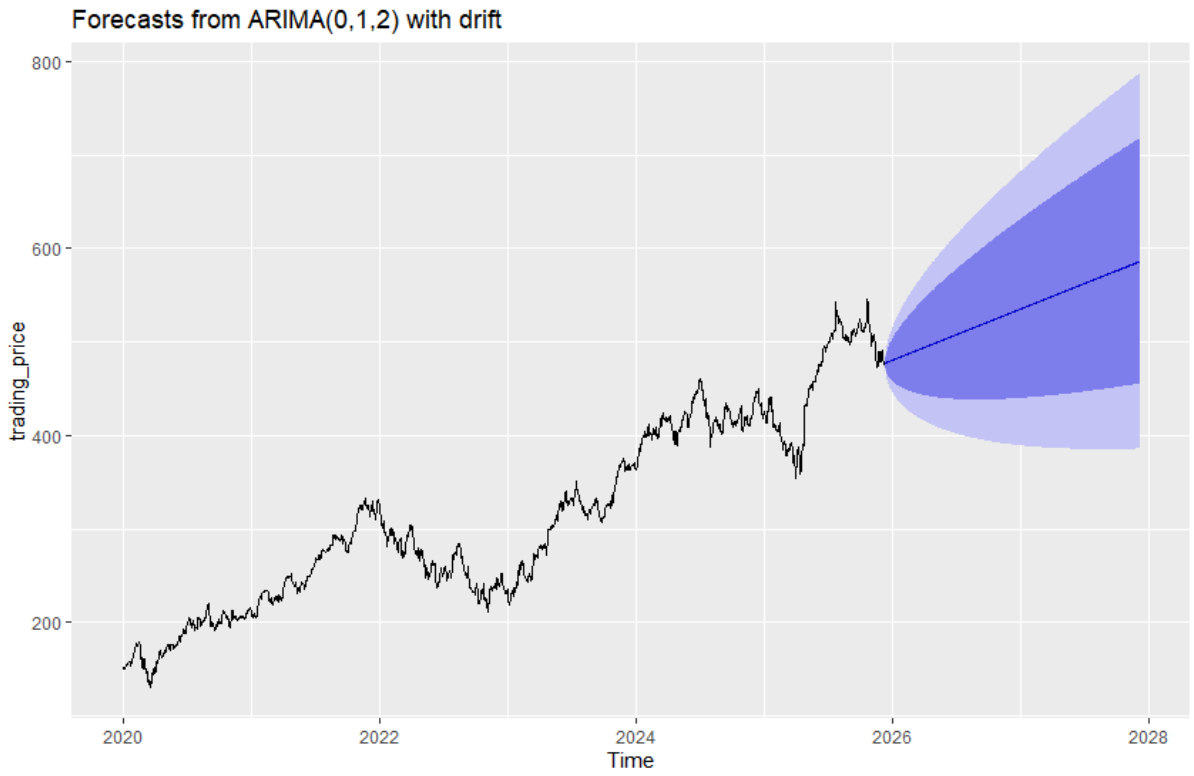
En este caso, vemos que el p-value es bastante mayor que 0.5, por lo que no podemos rechazar la hipótesis nula, por lo que no hay una correlación evidente, de manera que se cumple otro de los requisitos para la validación del modelo

Por último, para obtener la media residual, hacemos uso de uno de los valores que tiene el modelo (“$residuals”) y le hacemos la media a todos sus valores, y en este caso nos da una media muy cercana a 0

```
> print(mean(arima_automático$residuals))
[1] 0.0003044498
```

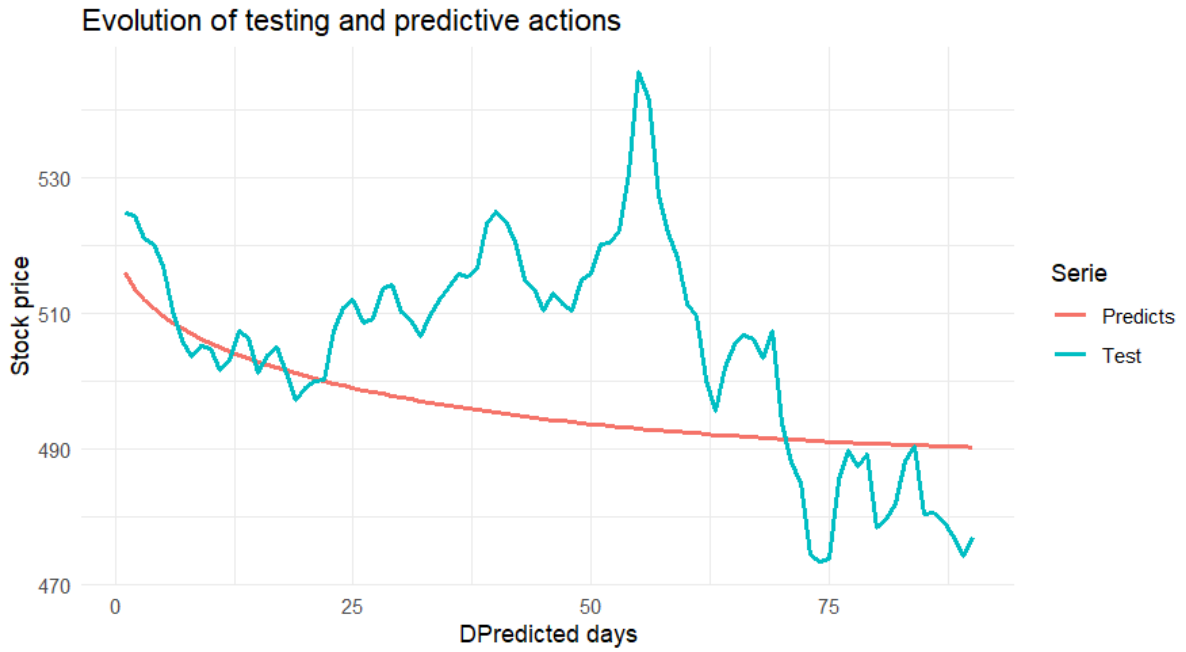
Por lo que se cumple el último requisito que nos faltaba para la validación del modelo

Por último, vamos a hacer predicciones con el modelo ya validado, en este caso, voy a hacer predicciones de los valores de las acciones de los próximos 500 días



Vemos que nos aparece una línea de la predicción y un intervalo de confianza por donde puede rondar los valores de la predicción

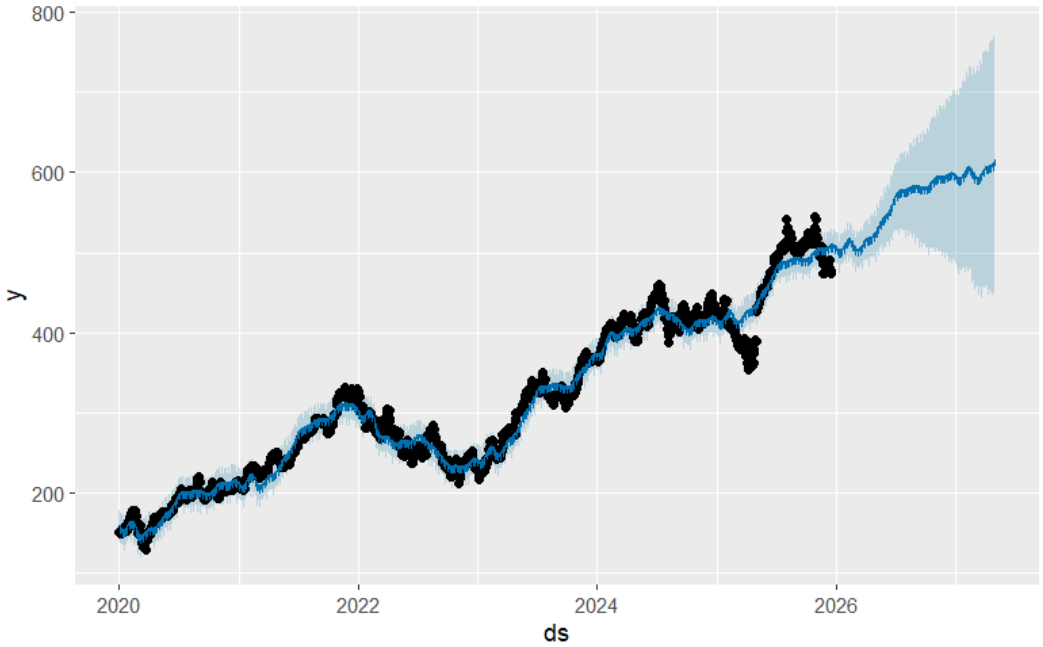
En cuanto a la ampliación, vamos a observar los resultados de los resultados de la predicción de los últimos tres meses y de los datos de test



Vemos gráficamente que, al principio sí que lo predice medianamente bien, luego hay un pico de subida en los datos de testing, por lo que en ese caso hay ciertos errores, pero luego cuando se vuelve a estabilizar, sí que lo predice bien de nuevo (días 70-85)

Al calcular el RMSE (nos permite medir la diferencia promedio entre valores predichos y reales) vemos que nos da un valor de **16.83**, es un valor algo alto, pero teniendo en cuenta que en los datos de testing hay un pico alto, no está del todo mal

Por otro lado, vamos a ver los resultados al entrenar el modelo de Facebook (Prophet), veamos gráficamente cual es el resultado:



Vemos que, a corto plazo, este último modelo es bastante mejor que el modelo ARIMA que se ha desarrollado en esta actividad ya que predice bastante bien, teniendo un intervalo de confianza bastante pequeño, pero pasado unos cuantos meses, vemos que el intervalo aumenta considerablemente, por lo que a largo plazo, ambos modelos son prácticamente iguales

Por último, vamos a ver los outliers que tiene nuestros datos y vamos a mostrarlo gráficamente:

Vemos que en este caso, como he mencionado anteriormente, no detecta ningún outliers

INTERPRETACIÓN DE LOS RESULTADOS

En conclusión, el filtrado de datos anteriores a 2020 ha servido bastante, ya que como he explicado anteriormente, los datos anteriores a 2020 no son representativos con el contexto actual de Microsoft. Si incluimos los datos anteriores de 2020, había mucho ruido y cambios bruscos, lo que hacía que el modelo no se ajustase bien. Sin embargo, al quitar dichos datos, podíamos hacer predicciones más realistas al contexto actual

Por otro lado, la serie sigue una tendencia claramente ascendente que, al descomponer la serie hemos podido ver su estacionalidad anual, donde caía a comienzos de año, y subía en verano

También, hemos visto que la serie no era estacionaria, esto se debe a que, al ser unos valores de acciones de una empresa, la evolución de estos se comportan como un random walk, algo habitual en precios de acciones

Por otra parte, en cuanto al modelo, en ambos casos (automático y manual), coincidía en sus parámetros, donde el parámetro d era igual a 1 para la diferenciación y el valor q era igual a 2. Pero el modelo automático tiene un valor de AIC algo menor, por lo que nos quedamos con ese modelo. Al ver luego los coeficientes de ma hemos visto que

- ➔ ma1: El error del periodo anterior influye ligeramente en el valor actual

- ➔ ma2: El error de dos periodos atrás no influye tanto en el valor actual

Al evaluar el modelo elegido, hemos podido ver los valores de sus errores, y son unos valores bastante aceptables, esto quiere decir que se ajusta muy bien a los datos históricos, al igual ocurre con el test de Ljung-Box, en el que lo pasa correctamente, no dejando ningún patrón sin explicar

Al evaluar el modelo y ver que es aceptable, podemos hacer predicciones, donde tiene un intervalo de confianza que cada vez se hace más grande, lo cual este modelo nos permite hacer predicciones a corto y medio plazo

En cuanto a los apartados de ampliación sacamos ciertas conclusiones

- ☐ Al predecir los últimos 3 meses y compararlos con los datos reales, vemos que si hay picos altos el error, como es obvio, aumenta bastante
- ☐ El modelo de Facebook (Prophet) es mejor que el modelo ARIMA a corto plazo, luego a medio-largo plazo ambos modelos tienen el mismo intervalo de confianza
- ☐ Este dataset de Microsoft no tiene al parecer ningún outliers desde 2020 hasta 2025